Parkinson’s Keystroke Screening ML Pipeline

Comprehensive Project Report

Project type: Explainable machine-learning research pipeline with a static browser inference demo
Dataset: PhysioNet neuroQWERTY MIT-CSXPD
Primary feature: Mean Absolute Consecutive Difference (MACD) of keystroke hold times
Primary model: One-feature logistic regression
Status: Full pipeline executed; 17 Python tests and 4 JavaScript parity cases passed

> **Important:** This project is a research and screening demonstration. It is not a medical diagnostic system and must not guide treatment decisions.

Executive summary

This project investigates whether keystroke timing can provide an interpretable signal for distinguishing Parkinson’s disease (PD) participants from control participants. The pipeline uses keyboard hold-time sequences from the PhysioNet neuroQWERTY MIT-CSXPD dataset, cleans malformed timing records, performs subject-level exploratory analysis, computes a paper-compatible MACD feature, evaluates the feature across cohorts, compares alternative baselines and feature families, tests timing robustness, and exports a small JSON model for browser inference.

The primary result is a pooled subject-level AUC of **0.8505**, with a subject-bootstrap 95% confidence interval of **0.7591–0.9302**. The two evaluation directions are **0.8222** for MIT-CS1PD → MIT-CS2PD and **0.8803** for MIT-CS2PD → MIT-CS1PD. The result is encouraging for an explainable research baseline, but the sample is small and the confidence interval is wide.

1. Project objectives

The project has five objectives. First, create a reproducible data-to-model pipeline rather than an analysis that depends on manually ordered notebook cells. Second, investigate whether successive hold-time changes differ between PD and control subjects. Third, replicate the simple MACD-style feature with transparent mathematics. Fourth, compare MACD with simple baselines, nQi-style windows, automated FRESH features, and classifier variants without

subject leakage. Fifth, export the selected model into a privacy-preserving browser demo that performs inference without storing typed text.

2. Dataset and study design

The input is the PhysioNet neuroQWERTY MIT-CSXPD dataset. The project uses two cohorts: MIT-CS1PD, representing the early-PD cohort, and MIT-CS2PD, representing the de novo/untreated cohort. In the executed run, the processed data contains 85 subjects and 116 valid sessions. Each session is associated with a participant ID, cohort, binary label, hold-time sequence, press-time sequence, keystroke count, and source file.

A **subject** is a participant; a **session** is one typing recording from that participant; a **cohort** is a study subgroup collected under one experimental context. Cross-cohort evaluation trains on one cohort and tests on the other, then swaps the direction. This is stricter and more informative than a random row split because the test group is a distinct cohort and participant identities are kept separate.

Shared session schema

```
session_id, subject_id, cohort, label, hold_times_s, press_times_s, n_keys
```

Hold times and stored JSON timings are measured in seconds. Browser event timestamps are measured in milliseconds and converted to seconds before feature calculation.

3. Pipeline architecture

The repository separates reusable scientific logic from executable entry points. The `src/` directory contains functions and model components that can be imported by scripts, tests, notebooks, or future services. The `scripts/` directory contains ordered commands that perform data preparation, EDA, feature extraction, evaluation, robustness analysis, comparison, and export. The notebook imports the same canonical functions and is an interactive reporting layer rather than a second implementation.

```
Raw PhysioNet archive
        ↓
01_prepare_data.py — parsing, labels, cleaning, session schema
        ↓
02_eda.py and 01_explore_data.ipynb — descriptive analysis
        ↓
03_extract_features.py — session and subject tables
        ↓
04_evaluate_macd.py — primary cross-cohort result
        ↓
05-14 — sensitivity, variants, baselines, FRESH, ablation, robustness
        ↓
15_export_model.py — macd_model.json
        ↓
Static browser inference demo
```

4. Data loading and cleaning

`src/data_loading.py` discovers cohort directories, reads ground-truth CSV files, maps participant IDs to labels, finds session files, and creates `Session` objects. `src/cleaning.py` applies the paper-compatible sanity checks. It rejects malformed numeric rows, invalid or extreme hold times, timestamp reversals, mouse events, long modifier keys, and backspace events. A cleaning audit is written to `data/processed/cleaning_summary.json` so the number and reason for removed rows are inspectable.

Cleaning occurs before feature extraction. This is important because a single timestamp error can create an artificial large consecutive change and distort MACD. The preparation step also enforces a minimum usable session length.

5. Expanded exploratory data analysis

The executed notebook `notebooks/01_explore_data.ipynb` contains 19 cells, including 9 executed code cells and 26 embedded output groups. It uses the canonical `src/eda.py` and `src/features.py` implementations. The automated script `scripts/02_eda.py` produces the official report and figures, while the notebook provides a richer interactive explanation.

The EDA covers dataset counts and label balance, cleaning quality, session length, hold-time quantiles and distributions, consecutive timing volatility, subject-level hold-time versus volatility, MACD distributions, corridor-pair counts, feature correlations, and violin plots. The key scatter

plot uses mean hold time on the x-axis and mean absolute consecutive change on the y-axis, with color indicating label and marker shape indicating cohort.

EDA interpretation

The EDA is intended to motivate a dynamic timing feature. If PD subjects have higher hold times, larger hold-time spread, or larger consecutive changes, then MACD is a reasonable candidate for further testing. However, the plots also show overlap between groups. They are descriptive evidence and quality checks, not causal evidence or clinical validation.

6. Feature engineering

Hold time

For each key event, hold time is calculated as release timestamp minus press timestamp. A session therefore becomes a sequence $x_1, x_2, \ldots, x_n$ measured in seconds.

MACD

For a session, the implementation calculates the lower and upper corridor quantiles. The primary setting is `ql = 0.60` and `qh = 1.00`, meaning the 60th percentile through the maximum. For every consecutive pair, both values must lie inside the inclusive corridor. MACD is the mean of the absolute differences for the retained pairs:

$$\mathrm{MACD} = \mathrm{mean}(|x_{i+1} - x_i|)$$

The feature is interpretable: it measures typical step-to-step hold-time change in a chosen portion of the participant's timing distribution. The JavaScript implementation uses the same linear quantile interpolation, inclusive bounds, pair mask, and NaN behavior as Python.

Simple features

The project also extracts mean hold time and hold-time standard deviation. These act as transparent baselines that help determine whether MACD adds information beyond a simple location or spread statistic.

nQi-style features

The nQi-style path divides press-time streams into non-overlapping 90-second windows and computes seven local timing features, including outlier proportion, distribution shape, mean flight

time, and coarse timing-bin proportions. A bootstrap ensemble of SVR-style models aggregates window scores. This path is intentionally more complex than the primary MACD model.

FRESH features

The FRESH path uses tsfresh to generate automated time-series features, ranks them using training-only labels, and evaluates logistic regression on the top `m` features. Feature selection and scaling occur inside the training side of each cohort direction to reduce leakage risk.

7. Evaluation design

The main evaluation is subject-safe cross-cohort evaluation. In one fold, all eligible MIT-CS1PD subjects are used for training and MIT-CS2PD subjects are held out for testing. In the reverse fold, the roles are swapped. A subject identifier intersection check prevents the same participant from appearing on both sides. Session-level predictions are averaged by subject before final AUC calculation.

The ROC AUC measures ranking ability: the probability that a randomly selected PD subject receives a higher score than a randomly selected control subject. AUC is threshold-independent, so the project also supports descriptive Youden cutoffs, confusion matrices, sensitivity, specificity, and balanced accuracy. The primary report emphasizes AUC and uncertainty because threshold choice should depend on the intended screening use.

Bootstrap confidence interval

The 95% interval is estimated by resampling subjects with replacement within class, recalculating AUC for each bootstrap sample, and taking the 2.5th and 97.5th percentiles. Subject-level resampling is important because multiple sessions from one participant are not independent people.

8. Primary result

Metric	Result
Pooled subject AUC	**0.8505**
Bootstrap 95% CI	**0.7591–0.9302**
MIT-CS1PD → MIT-CS2PD	**0.8222**
MIT-CS2PD → MIT-CS1PD	**0.8803**
MACD corridor	`0.60–1.00` quantiles
Minimum corridor pairs	`1`

The reported pooled value uses the common MACD feature ranking rather than pooling separately calibrated fold probabilities. This avoids letting independently fitted intercepts distort a single pooled ROC scale. The directional values remain essential because cohort transfer is asymmetric.

9. Baselines and model comparisons

Model or analysis	Pooled AUC / summary
Mean hold baseline	0.5565
Standard deviation baseline	0.5936
nQi-style ensemble	0.7447
MACD primary	**0.8505**
macd	0.8505
macd_log	0.8544
macd_norm	0.7436

The baseline comparison shows why MACD is useful as a compact candidate: it outperforms the simple mean-hold and standard-deviation baselines in the current executed result. The nQi-style

and FRESH paths provide more complex alternatives, but complexity alone is not evidence of clinical superiority.

FRESH curve

Top features `m`	AUC	CI low	CI high
1	0.7492	0.6406	0.8516
2	0.7458	0.6362	0.8406
3	0.7547	0.6462	0.8466
4	0.7353	0.6262	0.8333
5	0.7364	0.6262	0.8356
6	0.7458	0.6368	0.8433
7	0.7481	0.6401	0.8439
8	0.7436	0.6340	0.8400
9	0.7453	0.6345	0.8406
10	0.7569	0.6500	0.8505

The FRESH curve is an exploratory feature-selection analysis. It must not be interpreted as a final model-selection claim without stronger nested validation and an independent external dataset.

10. Robustness and sensitivity analysis

Corridor sensitivity

The lower quantile was swept across multiple values while keeping the upper bound at 1.0. Stability across this sweep indicates that results are not entirely dependent on one arbitrary corridor choice. Large changes would signal feature-design sensitivity.

`ql`	AUC
0.0	0.8228
0.4	0.8427
0.5	0.8571
0.6	0.8505
0.7	0.8400
0.8	0.8079

Keystroke-count curve

The pipeline evaluates truncated sessions at 50, 100, 150, 200, 300, 500, 1,000, and full length. This connects model quality to user burden in the future webapp.

Keystrokes	AUC
50	0.6999
100	0.7641
150	0.8029
200	0.8272
300	0.8405
500	0.8422
1000	0.8289
full	0.8505

Timing perturbations

Jitter, quantization, constant offsets, and global scaling approximate browser timing differences. Perturbations are applied to held-out sessions only. Offsets and positive scaling can preserve ranking by construction, so unchanged AUC does not mean the browser implementation is risk-free; feature-range warnings and quality checks are still required.

11. Model export and webapp architecture

The offline pipeline trains and validates the model. `scripts/15_export_model.py` then writes `models/macd_model.json`. The exported model contains the MACD corridor, logistic coefficient, intercept, training feature range, training quantiles, minimum keystroke requirement, units, source reference, and disclaimer. Current exported values include coefficient `0.5686` and intercept `0.0501`.

The browser does not rerun the entire research pipeline. It uses `webapp/js/capture.js` to collect timing values in memory, `webapp/js/macd.js` to compute the same MACD, and `webapp/js/app.js` to apply the JSON model. The logistic calculation is:

```
logit = intercept + coefficient × MACD
probability = 1 / (1 + exp(-logit))
```

The app never reads or stores the typed text itself. It is a research demo and is not validated on browser timing at clinical quality.

12. Software testing and reproducibility

The project currently passes 17 Python tests and four JavaScript parity cases. Tests cover cleaning, MACD edge cases, feature-table schemas, subject leakage protection, simple baselines, nQi windows, FRESH selection, robustness transformations, benchmark behavior, and Python/JavaScript MACD parity. Dependencies are pinned in `requirements.txt`, and `scripts/run_pipeline.sh` runs the ordered pipeline.

The fully executed EDA notebook contains embedded outputs and can be opened directly without rerunning cells. The official generated metrics are stored under `results/metrics/`, figures under `results/figures/`, and readable reports under `reports/`.

13. Work-package coverage

Work package	Coverage	Key files
WP1 Data and cleaning	Complete	src/config.py, src/data_loading.py, src/cleaning.py
WP2 EDA and feature tables	Complete	src/eda.py, src/feature_extraction.py, notebooks/01_explore_data.ipynb
WP3 MACD and variants	Complete	src/features.py, scripts/05_corridor_sensitivity.py, scripts/06_feature_variants.py
WP4 Simple baselines	Complete	src/baselines_simple.py, tests/test_baselines.py
WP5 nQi-style ensemble	Implemented approximation	src/baselines_nqi.py, tests/test_baselines_nqi.py
WP6 FRESH extraction/selection	Complete exploratory path	src/fresh_features.py, tests/test_fresh_selection.py
WP7 FRESH classifier/analysis	Complete exploratory path	src/fresh_eval.py, scripts/10_fresh_eval.py
WP8 Evaluation/integration	Complete	src/evaluation.py, src/compare.py, scripts/run_pipeline.sh
WP9 Ablation/robustness	Complete exploratory path	src/models.py, src/robustness.py, tests/test_robustness.py
WP10 Export/webapp	Complete research demo	src/export_model.py, models/macd_model.json, webapp/

The nQi implementation is explicitly an approximation of the described ensemble rather than a claim of exact reproduction of every nested-validation detail. FRESH and robustness results are exploratory and should be validated more rigorously before any deployment claim.

14. Limitations

The dataset contains only 85 subjects, so confidence intervals are wide and two cohort directions provide limited independent replication. The cohorts may differ in recruitment, device, timing logger, and participant characteristics. Browser timing is noisier and structurally different from the research logger. The current app displays a model output and research band, not a calibrated clinical probability. The pipeline does not establish fairness across demographic or device subgroups, does not include an external clinical dataset, and does not prove that MACD is specific to Parkinson's disease rather than correlated with other motor, cognitive, or typing factors.

15. How to execute the project

From the project root:

```
python3 -m pip install -r requirements.txt
./scripts/run_pipeline.sh
node tests/parity_check.mjs
```

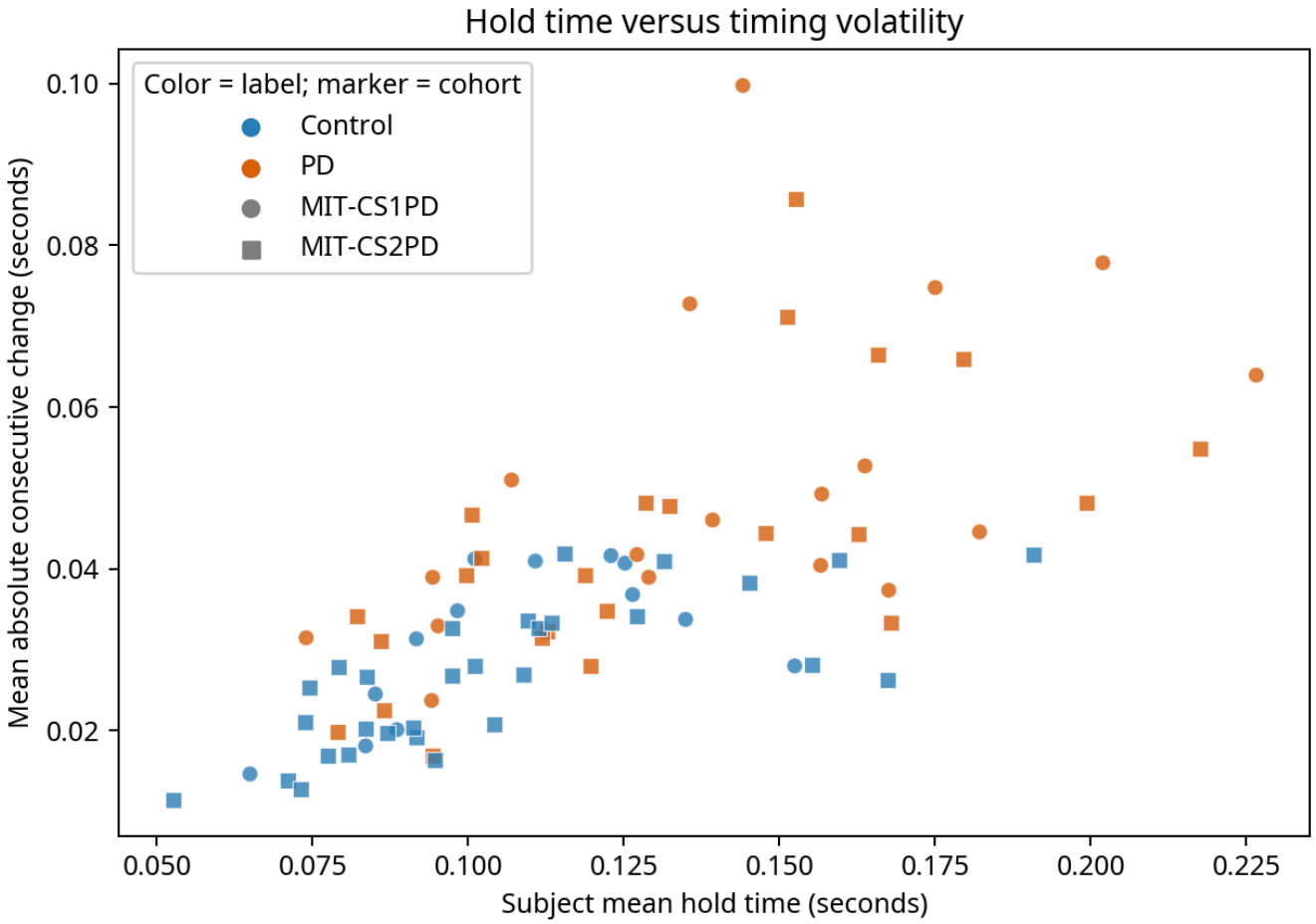
To open the webapp after export:

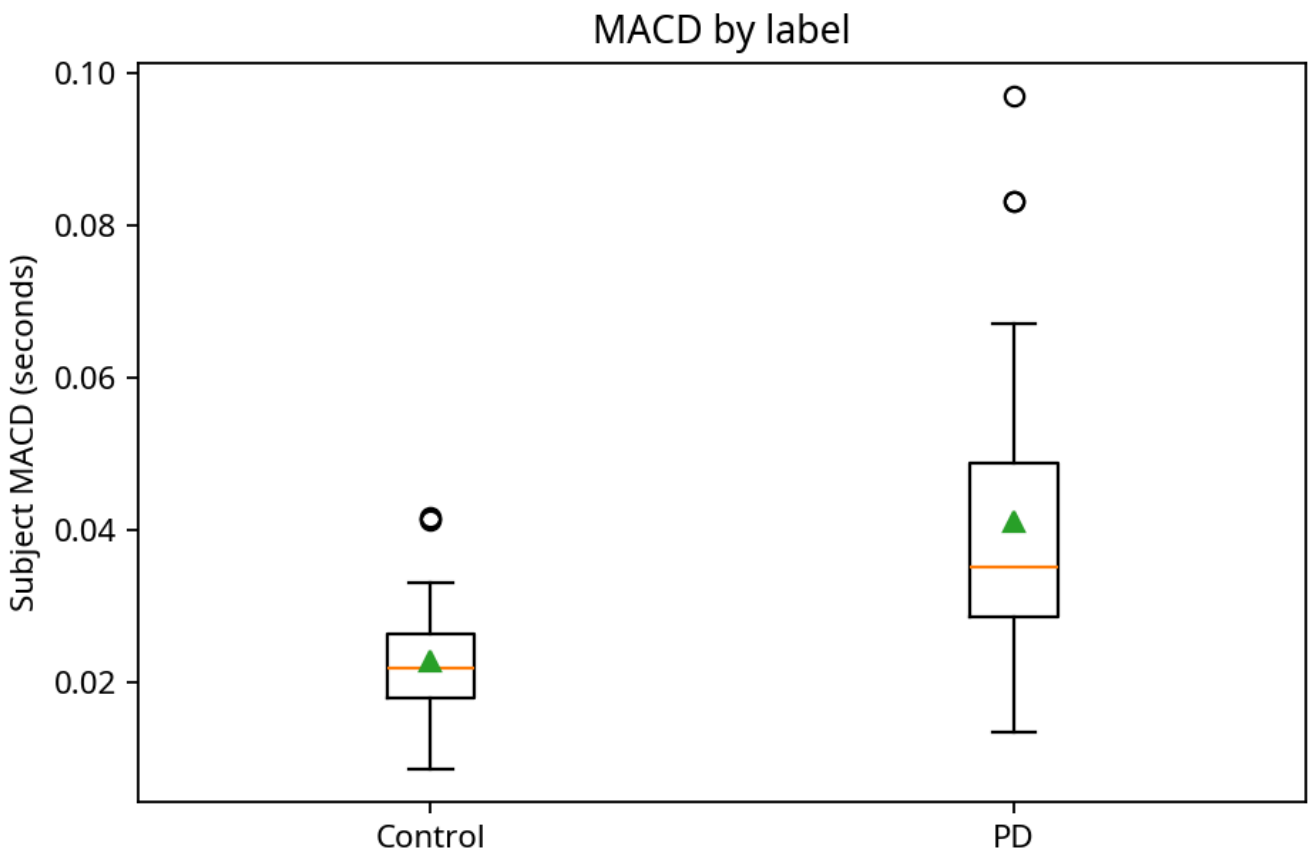
```
python3 -m http.server 8000
# open http://localhost:8000/webapp/
```

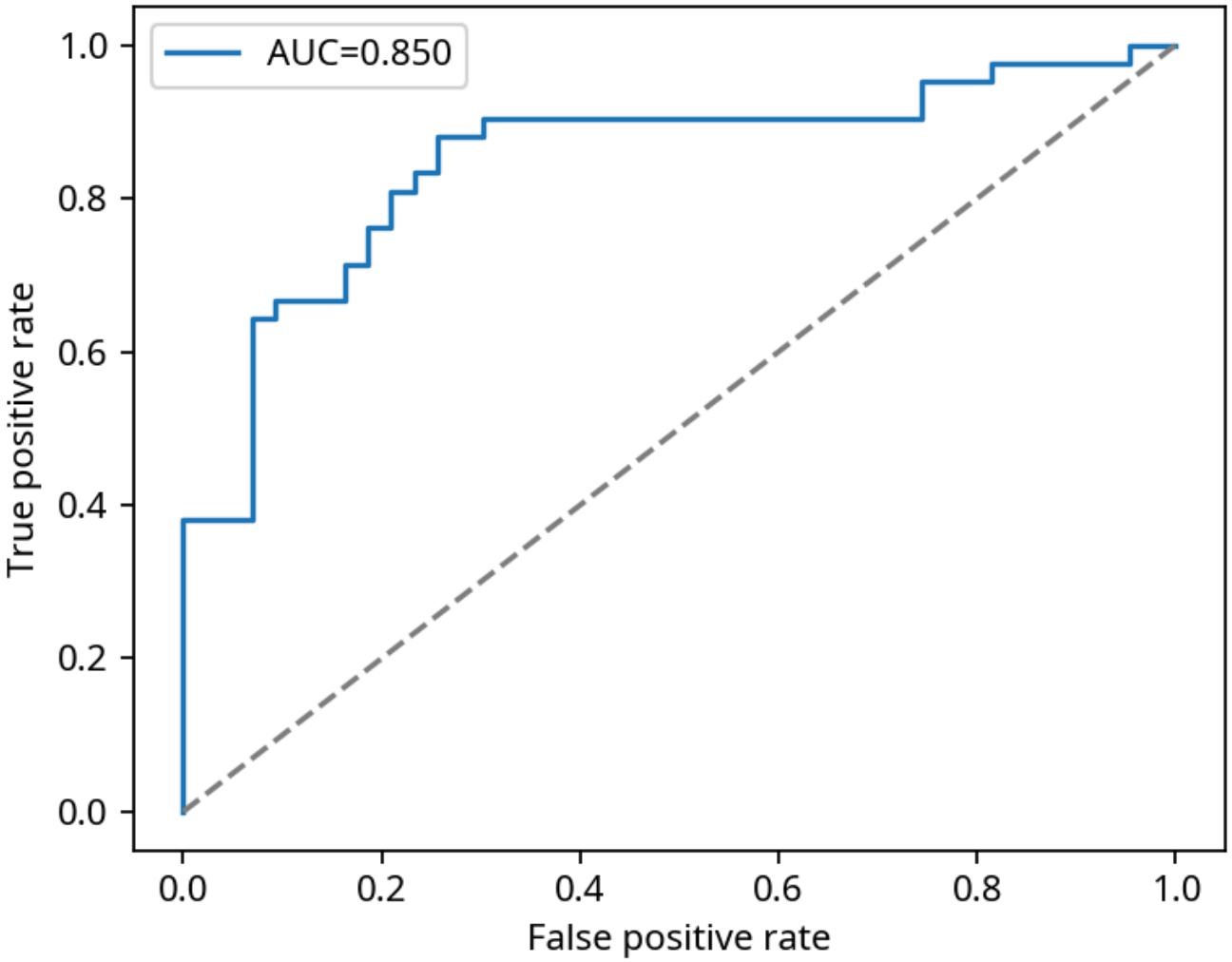
16. Deliverables and appendix

The principal deliverables are the source modules in `src/`, the executable stages in `scripts/`, the executed notebook in `notebooks/01_explore_data.ipynb`, the exported model in `models/macd_model.json`, the static webapp in `webapp/`, the generated metrics and figures, the test suite, and this report. The full inventory is maintained in `PROJECT_STRUCTURE.md`.

Selected figures




MACD by label
Subject MACD (seconds)
0.10
0.08
0.06
0.04
0.02
Control
PD



References

1. PhysioNet neuroQWERTY MIT-CSXPD dataset: https://physionet.org/content/nqmitcsxpd/
2. Milne, Farrahi, and Nicolaou, *Less is More: Univariate Modelling to Detect Early Parkinson's Disease from Keystroke Dynamics*.
3. Project-local executed metrics, notebook, source code, and generated reports.